**BIG DATA · Academic year 2026-2027**

# Assignment 1

## Basic Matrix Multiplication in Python, Java and C

Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria

Carlos Falcón Brito

**Repository:** https://github.com/Cafabri/Assigment1

## Contents

**1 Introduction and Objective** **2**

**2 Background** **2**

**3 Methodology** **2**

**4 Results** **4**

**5 Discussion** **4**

**6 Conclusions** **7**

# Abstract

This study compares a naive $O(n^3)$ triple-loop matrix multiplication implemented in Python 3.11, Java 21 (JVM with JIT) and C (gcc 16.1.0, `-O2`) across matrix sizes from $n = 10$ to $n = 1000$. Runtime grows cubically in all three languages. C is the fastest implementation at every tested size. Java takes approximately $2\times$ as long as C at $n = 1000$; Python reaches approximately 12 minutes at $n = 1000$, around $1500\times$ slower than C. Memory usage scales as $O(n^2)$ across all implementations. Results are scoped to the tested hardware, compiler settings and input sizes.


# 1 Introduction and Objective

Matrix multiplication is a fundamental operation in scientific computing. Its naive implementation requires $O(n^3)$ arithmetic operations for $n \times n$ matrices, making performance differences across languages clearly visible as $n$ grows.

**Research question:** How does matrix size affect the runtime and memory usage of a naive triple-loop matrix multiplication implemented in Python, Java and C, when each language runs in its standard execution environment?

**Hypothesis:** Runtime will grow cubically for all three implementations. C, compiled with standard optimisation flags, is expected to be the fastest. Java with JIT is expected to be close to C. Python, as an interpreted language, is expected to be one to two orders of magnitude slower than the compiled implementations.

# 2 Background

The standard triple-loop algorithm computes $C[i][j] = \sum_k A[i][k] \cdot B[k][j]$ for all $i$, $j$, performing $n^3$ multiply-accumulate operations. Its theoretical complexity is $O(n^3)$ in time and $O(n^2)$ in space for the output matrix.

The three languages differ fundamentally in how they execute code:

- **Python (CPython):** interpreted line by line. Each operation involves dynamic type checking and object overhead, making inner loops expensive.
- **Java (JVM):** compiled to bytecode and then JIT-compiled to native machine code at runtime. The JIT identifies hot loops and applies optimisations automatically after warm-up.
- **C (gcc `-O2`):** compiled ahead-of-time to native machine code with standard compiler optimisations, including register allocation and instruction scheduling.

# 3 Methodology

## Implementations

Each implementation uses the same `ijk` triple-loop order and 64-bit floating-point arithmetic (`float` in Python, `double` in Java and C). No optimised library calls are used for the multiplication itself. All three implementations run on a single thread; no internal

library threads are created. Storage layout differs across languages: Python uses a list of lists (each inner list is a separate heap object); Java uses `double[][]`, an array of independently allocated row arrays; C uses a flat one-dimensional array with index $i \cdot n + j$, providing truly contiguous row-major storage.

## Correctness

Before benchmarking, each implementation verifies three test cases: $A \times I = A$ (identity matrix), $A \times \mathbf{0} = \mathbf{0}$ (zero matrix), and a $2 \times 2$ product with known result ([[1,2],[3,4]] × [[5,6],[7,8]] = [[19,22],[43,50]]). All checks passed in all three languages.

The test inputs use small integers (values 0–8), for which floating-point multiply-accumulate operations produce exact results with no rounding error. Exact equality (`==`) is therefore used as the verification criterion. For general random floating-point inputs, differences in evaluation order may introduce rounding errors bounded by machine epsilon ($\varepsilon \approx 2.2 \times 10^{-16}$ for IEEE 754 double precision); these do not affect the benchmarks since correctness is verified only on the integer-valued test cases.

## Execution environment

| Item | Value |
|---|---|
| CPU | Intel Core i7-13700KF (16 cores, 24 logical processors) |
| RAM | 31.8 GB |
| OS | Windows 11 Pro |
| Python | CPython 3.11.9 — interpreter, no JIT |
| Java | OpenJDK 21.0.12 (Microsoft Build) — JIT enabled (default) |
| C compiler | gcc 16.1.0 (MSYS2/MinGW64), flags: `-O2` |

Table 1: Execution environment.

## Budget

A maximum runtime of 15 minutes per run and a memory limit of 30 GB were declared before testing large sizes. All configurations up to $n = 1000$ completed within these limits. $n = 1000$ is both the largest attempted and the largest feasible size under the declared budget.

## Measurement protocol

- **Sizes tested:** $n \in \{10, 50, 100, 200, 300, 500, 750, 1000\}$.
- **Repetitions:** 5 measured runs per configuration, preceded by warm-up runs (1 for Python, 3 for Java, 2 for C) to reduce cold-start effects.
- **Timing:** `perf_counter` (Python), `nanoTime` (Java), `QueryPerformanceCounter` (C). All timers measure wall-clock time of the multiplication kernel only; input generation is outside the timed region.

- **Memory:** peak heap via `tracemalloc` (Python); JVM heap delta before and after `gc()` (Java); allocated matrix storage $3n^2 \times 8$ bytes (C — exact, since allocation is explicit).
- **Summary statistic:** median and interquartile range (IQR) over 5 repetitions.
- **Inputs:** matrices generated with fixed seeds (42 for A, 43 for B) using each language's standard PRNG, values in $[0, 10]$. Cross-language PRNGs differ, so matrices are statistically similar but not bit-identical.

# 4 Results

## Runtime

Table 2 reports the median runtime and Table 3 the IQR for all configurations. Figure 1 shows the growth on a log-log scale; dotted lines show the cubic model $t \propto n^3$ anchored at each language's smallest measurable point. The C result at $n = 10$ is below the timer resolution and is omitted from the plots.

| n | C (s) | Java (s) | Python (s) |
|---|---|---|---|
| 10 | <0.000001 | 0.000022 | 0.000202 |
| 50 | 0.000035 | 0.000140 | 0.028243 |
| 100 | 0.000309 | 0.000444 | 0.344134 |
| 200 | 0.002731 | 0.003165 | 1.801821 |
| 300 | 0.010202 | 0.013094 | 11.699590 |
| 500 | 0.048281 | 0.085622 | 58.140186 |
| 750 | 0.177874 | 0.365947 | 284.902533 |
| 1000 | 0.480131 | 0.989217 | 721.430916 |

Table 2: Median runtime (seconds) over 5 repetitions per configuration.

| n | C IQR (s) | Java IQR (s) | Python IQR (s) |
|---|---|---|---|
| 50 | 0.000003 | 0.000095 | 0.000358 |
| 100 | 0.000151 | 0.000008 | 0.000714 |
| 200 | 0.000006 | 0.000067 | 0.015807 |
| 300 | 0.000133 | 0.000248 | 0.026359 |
| 500 | 0.000307 | 0.000559 | 0.380738 |
| 750 | 0.000321 | 0.001647 | 0.958414 |
| 1000 | 0.001999 | 0.051162 | 1.752245 |

Table 3: IQR of runtime (seconds) over 5 repetitions per configuration.

## Memory

# 5 Discussion

**Cubic growth.** All three implementations exhibit clear cubic growth. From $n = 100$ to $n = 200$ (factor of 2), runtimes multiply by approximately 8 ($2^3$), consistent with $O(n^3)$.

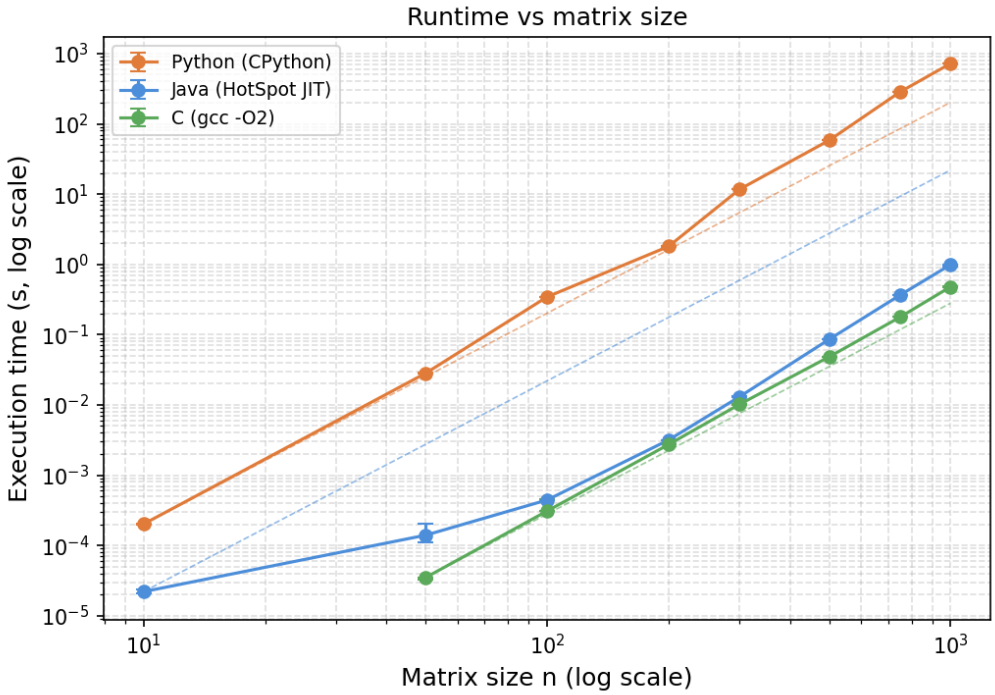



Figure 1: Median runtime vs matrix size $n$ (log-log scale). Dotted lines show the cubic model $t \propto n^3$ anchored at each language's first measurable point. Error bars show IQR.

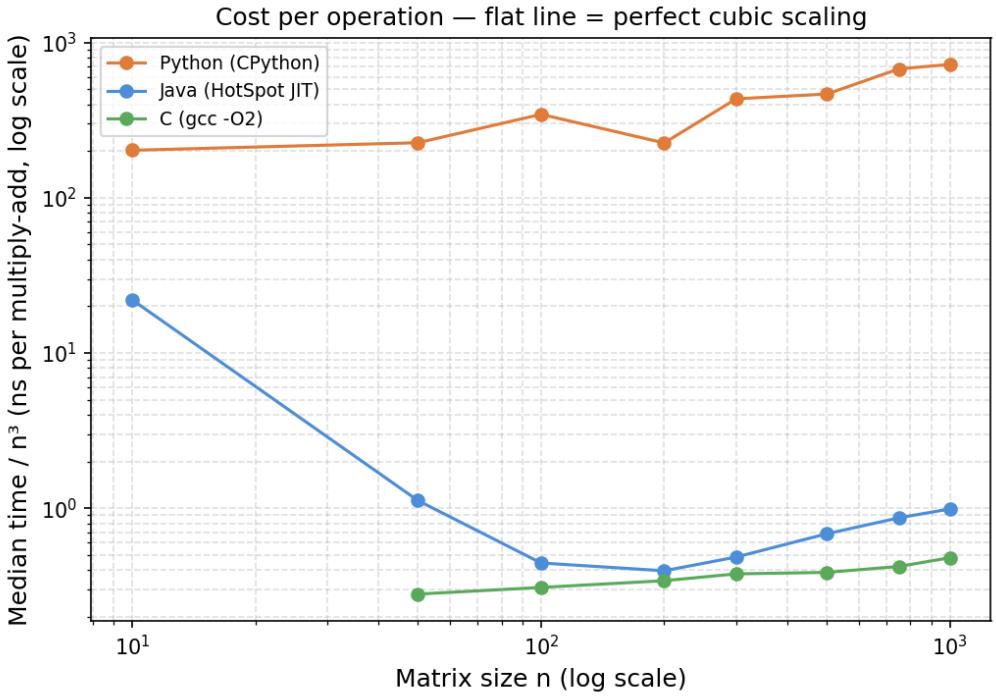



Figure 2: Median time divided by $n^3$ (nanoseconds per multiply-add). A flat line indicates perfect cubic scaling; an upward trend reveals when cache pressure makes each operation more expensive.

| n | C (KB) | Java (KB) | Python (KB) |
|---|---|---|---|
| 10 | 2.3 | 41.6 | 1.2 |
| 100 | 234.4 | 86.9 | 312.5 |
| 300 | 2109.4 | 747.8 | 2825.1 |
| 500 | 5859.4 | 2008.4 | 7837.7 |
| 1000 | 23437.5 | 7952.3 | 31307.1 |

Table 4: Median memory usage (KB). Metrics differ across languages; see Section 3.

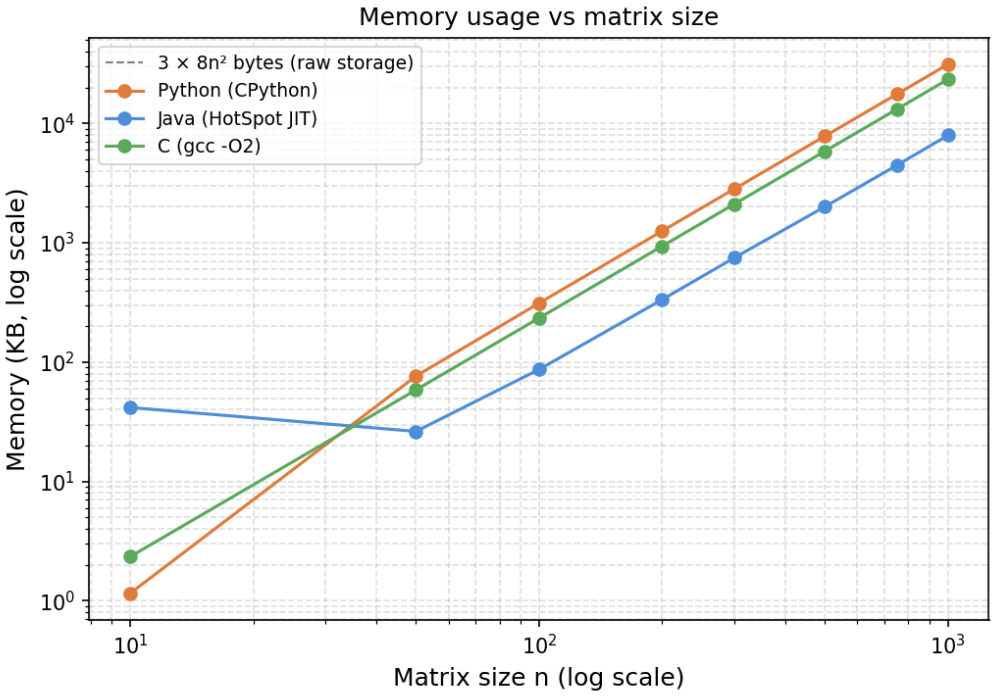



Figure 3: Memory usage vs matrix size $n$ (log-log scale). The dashed line shows the raw storage of three float64 matrices ($3n^2 \times 8$ bytes).

Figure 2 shows the cost per inner-loop operation (median time divided by $n^3$): a flat line would indicate perfect cubic scaling. For all three languages the cost per operation rises at large $n$, consistent with cache pressure as the working set exceeds the L2/L3 cache.

**C is the fastest.** C compiled with `-O2` is the fastest implementation at every tested size. At $n = 1000$, Java takes approximately $2\times$ as long as C, and Python takes approximately $1500\times$ as long. The gap between C and Java is explained by the difference in storage layout: Java's `double[][]` stores each row as a separate heap object, so the inner loop accessing column $j$ of $B$ must first load a row reference, adding an indirection that C's contiguous flat array avoids.

**Python overhead.** Python is approximately $1500\times$ slower than C and $730\times$ slower than Java at $n = 1000$. CPython interprets each bytecode instruction individually, performs dynamic type checks on every operation, and represents numbers as heap-allocated objects. These overheads dominate the inner loop and explain the large constant factor relative to compiled implementations.

**Memory.** Memory scales as $O(n^2)$ in all three cases, consistent with storing three $n \times n$ matrices. The different measurement methods produce different absolute values: C reports exact allocated storage, while Python and Java include object headers and runtime overhead. These metrics should not be compared as if identical.

**Limitations.** Cross-language PRNG differences mean input matrices are statistically similar but not bit-identical. The Java memory metric is approximate due to GC activity. The C timer resolution is insufficient to measure $n = 10$ (result below 1 $\mu$s). Results are specific to the tested hardware, OS and language versions.

# 6 Conclusions

All three implementations exhibit the expected $O(n^3)$ runtime growth. Under the tested conditions — each language in its standard execution environment — C (`-O2`) achieves the lowest execution time, followed by Java (JIT), then Python. Python's interpreter overhead makes it unsuitable for large-scale naive matrix multiplication without vectorised libraries. Memory usage scales quadratically in all cases, with differences in absolute values explained by the measurement method.

These conclusions apply strictly to the tested implementation, hardware and settings. Cache blocking, loop reordering and further compiler optimisations will be explored in Assignment 2.

# Generative AI Disclosure

**Tool:** Claude Code (Claude Sonnet 4.6, Anthropic).

**Purpose and affected parts:**

- *Planning and structure* — the experimental plan and project organisation were designed with AI assistance.

- *Implementation scaffolding* — the structure of the Python, Java and C benchmark files (timing harness, CSV output, warm-up loop) was generated with AI support and reviewed and validated by the author.
- *Debugging* — AI assistance identified and fixed two issues: a Java locale bug producing malformed CSV output, and the unavailability of `timespec_get` in MinGW, replaced with `QueryPerformanceCounter`.
- *Analysis script* — the Python plotting script was generated with AI support and reviewed by the author.
- *Report drafting and language revision* — the structure and initial text of this report were produced with AI assistance. All technical claims, measurements and conclusions were verified by the author.

**Author's responsibility:** The author understands all implementation code, validated the correctness checks, verified that the raw measurements are consistent with the expected $O(n^3)$ behaviour, and is able to reproduce and defend every experiment described in this report.

# References and Materials


- Source code, raw measurements (CSV), analysis scripts and plots: https://github.com/Cafabri/Assigment1
- Python Software Foundation. *Python 3.11 documentation*. https://docs.python.org/3.11/
- Oracle. *Java SE 21 Documentation*. https://docs.oracle.com/en/java/javase/21/
- Free Software Foundation. *GCC 16 Manual*. https://gcc.gnu.org/onlinedocs/
- Cormen, T.H., Leiserson, C.E., Rivest, R.L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.